

Physics-Based Frisbee Controller with Dynamic Cinematic Camera

Overview

This skill assessment is a small Unity gameplay system extracted from a larger work-in-progress PC frisbee game. The control system is primarily inspired by Flick Soccer, where the player aims and shoots quickly, then watches the trajectory and impact play out.

My twist was to take that quick aim-and-shoot idea and apply it to a frisbee, putting more emphasis on the satisfaction of curving around obstacles. The project is built around two modular systems:

1. A physics-based frisbee controller with lock-on targeting, charged throws, directional influence, and curved auto-guidance.
2. A dynamic camera system with a precise aiming mode and a cinematic helicopter mode for high-speed action moments.

Functional Overview

The system is designed as a cooperation between the player controller, camera system, targets, feedback layer, and level structure.

The player controller takes user input and controls the frisbee's movement: throw power, speed, distance traveled, turning behavior, and how curving feels. It also manages the player's flow between throwing, flying, and dead states.

The camera system has two main functions: precise third-person aiming and a cinematic helicopter camera for flight. The helicopter camera switches between shooting angles and distances to keep high-speed movement readable and dramatic. It generates possible angles, checks how clear each shot is by projecting along the frisbee's future trajectory, then chooses a usable view. Designers can tune how often the camera switches angles. Camera distance is also adaptive: it increases when the camera faces away from the flight path so the player stays oriented, and it can scale with frisbee speed through an exposed multiplier.

Targets provide points that the frisbee can aim toward, lock onto, and curve around. They also refill the player's throws, and can be adapted to trigger level endings, level changes, or other programmed events. The feedback layer provides particles, sounds, UI, and impact responses when prompted by the player, movement, and target systems.

The level is built from forms, planes, and wide or narrow obstacles that vary in elevation, shape, and distance. These elements create a course that challenges the player to use the frisbee's curve, speed, and throw chaining, or to slow down depending on the type of challenge.

Architectural Overview

Input and Player Coordination

PlayerInputController takes input using Unity's input system and manages input buffers, such as throw release and switching back into throwing.

PlayerController owns the main gameplay decisions. It reads input state, prompts state changes, responds to collisions, manages throw count, and holds global player settings such as starting throws and throws gained from targets.

State Flow

PlayerStateController runs the state machine. It safely exits and enters states by popping and pushing them on a state stack, allowing enter and exit behavior.

ThrowingState, FlyingState, and DeadState decide which systems should run every frame. This keeps the player flow readable: throwing runs aiming and charging, flying runs turning and low-speed checks, and dead releases the frisbee from normal control.

Frisbee Movement

FrisbeeMovementController handles the Rigidbody and the main frisbee tuning. It applies force for throwing, turning, curving, vertical adjustment, and reset movement. This is where speed, throw power, curve feel, and turning behavior are tuned.

FrisbeeCollision forwards collision and trigger events from the frisbee object back to the player gameplay layer.

Targeting

TargetSelector uses camera information to create an arcade-style lock-on system, choosing the most centered valid target on screen. It stores both the aim target and the turning target.

TargetCollisionHandler represents targets that can be hit or broken. It provides target-side collision and break information, which the player and feedback systems can respond to, such as refilling throws, clearing targets, or triggering level-specific events.

Camera

PlayerCameraController switches between the third-person aiming camera and the helicopter camera used during flight. FreeLookVelocityAngleSetter supports precise aiming by preparing the third-person camera around the frisbee's movement direction. HeliCameraDirectionSetter generates possible helicopter camera angles, evaluates shot clearance along the frisbee's projected movement, and adjusts camera distance for readability at speed.

Feedback and UI

PlayerFeedback owns player-side feedback such as throw sounds, charge sound, flying sound, and charge light. TargetFeedback owns aim target and turning target visual effects. AimVisual updates the aiming arrow and throw-power particles. PlayerStatsUI displays remaining throws by enabling throw icons consecutively.

Player Flow

1. The player moves the camera toward the general direction of their desired target.
2. The target selector chooses the most centered valid target on screen.
3. The player holds the action button to charge throw strength.
4. The player can hold left or right to influence the throw direction and curve.
5. On release, the frisbee launches.
6. While flying, the frisbee can curve toward the turning target if there is enough distance and speed.
7. Hitting a target refills throws and lets the player continue chaining shots.

Key Decisions

Live Force-Based Curving

The frisbee curves toward targets by applying a designer-tuned force until its movement aligns with the target. I considered other arcing methods, such as calculating the curve beforehand, but chose this approach because it keeps the frisbee feeling physical and responsive. The curve is not pre-baked; the frisbee adjusts live. This also creates skill expression: players can learn how fast the frisbee curves, use speed and throw angle to shape wide arcs, and judge the distance needed to actually hit a target.

Single State Machine

The player flow is organized around a single state machine instead of many separate rules checking when each behavior is allowed. The project could have worked either way, but I chose this structure to make the player's current mode explicit: throwing, flying, or dead. Each state decides which systems run during that mode, which makes behavior easier to reason about and avoids scattering condition checks across many scripts. It also let me practice using a state stack with safe enter and exit behavior.

Cinematic Flight Camera

The cinematic camera went through several iterations. Earlier versions used stickiness to avoid too many small angle changes. The current version instead generates new camera angles within timed intervals, while making sure each angle is meaningfully different from the previous one. The camera evaluates how open each angle is by checking ahead in the frisbee's flight direction, then chooses a cleaner view. This is supported by Cinemachine obstacle avoidance, which smooths around unavoidable obstructions created by the level geometry.

Modularity

The project is organized so the controller and camera can be understood as separate but cooperating systems. The controller side is split into input, player coordination, state flow, movement, targeting, and feedback. This keeps the main player rules separate from physics tuning, target selection, and presentation.

The camera side is also modular. PlayerCameraController exposes a small API for switching camera modes, while the third-person aiming camera and helicopter camera behavior are tuned through their own scripts and Cinemachine settings. The camera does not own the frisbee rules; it reads movement and spatial information to support control, readability, and cinematic presentation.

The Git repository contains the Unity project and should include setup instructions for opening and running the project.